UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

COORDENADORIA DO CURSO DE ENGENHARIA DE SOFTWARE

MATHEUS DE AQUINO DA SILVA

PROPOSTA DE MECANISMO DE BUSCA SEMÂNTICA PARA APLICAÇÕES WEB COM ARMAZENAMENTO VETORIAL EM POSTGRESQL

RELATÓRIO TÉCNICO PARCIAL

TCC 1

DOIS VIZINHOS

2026

MATHEUS DE AQUINO DA SILVA

PROPOSTA DE MECANISMO DE BUSCA SEMÂNTICA PARA APLICAÇÕES WEB COM ARMAZENAMENTO VETORIAL EM POSTGRESQL

Relatório técnico apresentado como requisito parcial à obtenção do título de Bacharel em Engenharia de Software, da Universidade Tecnológica Federal do Paraná.

Orientador: Prof. Dr. Bruno Honorato

DOIS VIZINHOS

2026

LISTA DE ILUSTRAÇÕES

Figura 1 – Arquitetura de componentes e infraestrutura da solução.....6
Figura 2 – Diagrama de fluxo de dados e orquestração da busca semântica.....6

LISTA DE QUADROS

Quadro 1 - Matriz de requisitos funcionais do mecanismo de busca.....8
Quadro 2 - Matriz comparativa de trabalhos relacionados.....12
Quadro 3 - Dicionário de dados das tabelas de persistência vetorial.....14
Quadro 4 - Cronograma de atividades projetado para o TCC 2.....18

LISTA DE SIGLAS

ANN	Approximate Nearest Neighbors
API	Application Programming Interface
CPU	Central Processing Unit
CSV	Comma-Separated Values
DDL	Data Definition Language
DML	Data Manipulation Language
HNSW	Hierarchical Navigable Small World
ID	Identifier
MMR	Mean Reciprocal Rank
PLN	Processamento de Linguagem Natural
RF	Requisito Funcional
RI	Recuperação da Informação
SGBD	Sistema de Gerenciamento de Banco de Dados
SQL	Structured Query Language
TCC	Trabalho de Conclusão de Curso
TF-IDF	Term Frequency-Inverse Document Frequency
TI	Tecnologia da Informação

LISTA DE ACRÔNIMOS

BERT	Bidirectional Encoder Representations from Transformers
BM25	Best Matching 25
FAQ	Frequently Asked Questions
JSON	JavaScript Object Notation

SUMÁRIO

1 INTRODUÇÃO..... 5
1.1 Objetivo Geral..... 5
1.2 Objetivos Específicos..... 5
1.3 Descrição da Regra de Negócio..... 6
1.4 Justificativa/Contextualização..... 6
2 DESENVOLVIMENTO..... 6
2.1 Especificação das Funcionalidades da Ferramenta..... 6
2.1.1 Módulo de Ingestão e Pré-processamento de Dados..... 8
2.1.2 Módulo de Vetorização e Persistência Vetorial..... 9
2.1.3 Módulo de API e Orquestração de Consultas (Interface de Aplicação)..... 9
2.2 Fundamentação Teórica em Recuperação da Informação e Modelagem Vetorial..... 10
2.2.1 Busca Lexical versus Busca Semântica..... 11
2.2.2 Fundamentos de Embeddings e Modelos Sentence-Transformers..... 11
2.2.3 Formalização das Métricas de Avaliação..... 12
2.3 Trabalhos Relacionados..... 12
2.4 Persistência de Dados..... 14
2.5 Tecnologias Utilizadas..... 16
2.6 Desenho Experimental e Protocolo de Avaliação..... 17
2.6.1 Definição da Amostra e Conjunto de Dados..... 17
2.6.2 Estabelecimento do Padrão-Ouro (Ground Truth)..... 18
2.6.3 Pipeline de Execução dos Testes..... 18
2.7 Etapas e Cronograma de Desenvolvimento..... 19
3 CONSIDERAÇÕES FINAIS..... 20

1 INTRODUÇÃO

O suporte técnico corporativo, comumente denominado *Help Desk*, enfrenta desafios crescentes na gestão e recuperação de informações em bases de conhecimento. Com o aumento do volume de dados não estruturados, a eficácia do atendimento depende diretamente da precisão das ferramentas de busca disponíveis para técnicos e usuários finais. Tradicionalmente, esses sistemas operam via busca lexical, baseada na correspondência exata de palavras-chave, o que frequentemente resulta em falhas de recuperação quando há uma disparidade entre a linguagem técnica dos manuais e a linguagem coloquial dos usuários. Nesse contexto, este projeto propõe o design de um mecanismo de busca semântica fundamentado em representações vetoriais (*embeddings*). Ao contrário da busca tradicional, a abordagem semântica mapeia o significado contextual das consultas, permitindo que o sistema compreenda a intenção do usuário mesmo diante de variações linguísticas ou sinônimos. A solução utiliza a infraestrutura do banco de dados PostgreSQL com a extensão *pgvector*, integrando capacidades de inteligência artificial a um ambiente de persistência de dados robusto e consagrado na engenharia de software. O escopo deste trabalho delimita-se à proposição e modelagem da arquitetura de busca, definindo o fluxo de ingestão, vetorização e recuperação de dados. Embora um protótipo funcional seja utilizado como prova de conceito para validar a viabilidade técnica, o foco primordial reside na especificação da solução e na definição de métricas de avaliação que permitam comparar o desempenho do modelo semântico frente às abordagens convencionais de recuperação da informação.

1.1 Objetivo Geral

Propor e modelar um mecanismo de busca semântica para bases de conhecimento de *Help Desk*, utilizando armazenamento vetorial em PostgreSQL, visando estabelecer uma arquitetura escalável e eficiente para a recuperação de informações baseada em contexto.

1.2 Objetivos Específicos

- Investigar técnicas de representação vetorial e modelos de *embeddings* adequados para o domínio de suporte técnico;
- Modelar a *pipeline* de dados, abrangendo as etapas de ingestão, pré-processamento e vetorização de documentos;
- Projetar a arquitetura de persistência, especificando o uso de índices e funções de similaridade no PostgreSQL com *pgvector*;
- Especificar as interfaces de uma API para orquestrar o fluxo entre as consultas dos usuários e a base vetorial;
- Definir uma metodologia de avaliação rigorosa baseada em métricas de precisão, revocação e *Mean Reciprocal Rank* (MRR) para validação do experimento.

1.3 Descrição da Regra de Negócio

O projeto propõe a modelagem de um ecossistema de recuperação de informações voltado para ambientes de suporte técnico (*Help Desk*). Atualmente, a eficiência operacional desses setores está vinculada à agilidade com que técnicos ou usuários finais localizam soluções em bases de conhecimento, como manuais e *FAQs*. Entretanto, sistemas baseados em busca lexical (palavras-chave) apresentam limitações severas ao exigir correspondência exata de termos técnicos. A regra de negócio central desta proposta consiste na especificação de um sistema capaz de interpretar a intenção do usuário, superando as barreiras da linguagem coloquial ou não técnica. Como requisito fundamental, ao receber uma consulta como "minha internet não funciona", a arquitetura deve ser capaz de mapear e recuperar documentos técnicos pertinentes a "falhas de conectividade" ou "configurações de *gateway*". O escopo define que o sistema deve atuar como uma ferramenta de apoio cognitivo, reduzindo o tempo médio de atendimento e aumentando a autonomia do usuário.

1.4 Justificativa/Contextualização

A relevância deste projeto fundamenta-se na crescente ineficiência dos métodos de busca convencionais perante o volume exponencial de dados não estruturados na Engenharia de Software. Em ambientes de serviços, o tempo de resposta é uma métrica crítica de sucesso, e a falha na recuperação da informação impacta diretamente a viabilidade do suporte. Segundo Pressman e Maxim (2021), a qualidade de um produto de software é aferida por sua eficácia em resolver problemas reais; esta ferramenta justifica-se pela aplicação de modelos de Processamento de Linguagem Natural (PLN) para converter textos em espaços vetoriais multidimensionais, onde a similaridade de significado é calculada pela proximidade geométrica. Academicamente, o trabalho contribui para o estudo de tecnologias emergentes como o *Sentence-Transformers* e o uso do PostgreSQL com *pgvector* como solução unificada de persistência e busca vetorial. Do ponto de vista da Engenharia de Dados, essa escolha arquitetural evita a fragmentação do sistema em múltiplas infraestruturas complexas. O rigor científico da proposta é estabelecido pela definição de métricas clássicas de avaliação de sistemas de busca, como Precisão, Revocação e *Mean Reciprocal Rank* (MRR), garantindo que a validação do experimento atenda aos padrões técnicos de recuperação da informação.

2 DESENVOLVIMENTO

2.1 Especificação das Funcionalidades da Ferramenta

A especificação de requisitos e o design arquitetural constituem etapas críticas no ciclo de vida do desenvolvimento de sistemas, estabelecendo as bases para a

manutenibilidade, modelagem e validação do produto de software (PRESSMAN; MAXIM, 2021). Para mitigar a lacuna semântica entre as consultas coloquiais dos usuários e os documentos técnicos, a ferramenta foi concebida sob uma abordagem modular baseada em uma Arquitetura em Camadas (*Layered Architecture*), garantindo o baixo acoplamento e a alta coesão entre os componentes do sistema.

A organização estrutural da solução proposto para o ambiente de *Help Desk* é segmentada em três subsistemas ou macrocomponentes principais, cujas responsabilidades técnicas são delimitadas a fim de sustentar o fluxo do sistema:

- **Subsistema de Ingestão e Processamento de Dados (Camada de Dados):** Responsável de forma exclusiva pela extração de arquivos brutos (JSON/CSV), execução de pipelines de higienização de texto e aplicação de algoritmos de quebra estruturada (*chunking*) para a preservação do contexto local de manuais.
- **Subsistema de Inteligência e Vetorização (Camada de Negócio):** Atua no mapeamento topológico das strings textuais para o espaço vetorial multidimensional. Suas responsabilidades incluem o gerenciamento do modelo *Sentence-Transformers* em memória e a transformação das requisições em tensores numéricos.
- **Subsistema de Orquestração e Interface (Camada de Aplicação/API):** Gerenciado pelo *framework* assíncrono *FastAPI*, este componente atua como o mediador central do sistema. É responsável por expor os *endpoints* de busca, interceptar as requisições dos usuários, acionar o subsistema de inteligência, disparar as consultas espaciais via SQL no *PostgreSQL* e aplicar os critérios de ordenação para a entrega do resultado final (*Top-N*).

A fim de evidenciar o funcionamento da solução sob perspectivas complementares, a Figura 1 ilustra a arquitetura estática global do ecossistema e o acoplamento estrutural dos componentes dentro do ambiente virtualizado. Na sequência, a Figura 2 detalha o fluxo de controle dinâmico e a interação temporal entre esses subsistemas durante a orquestração de uma requisição de busca semântica.

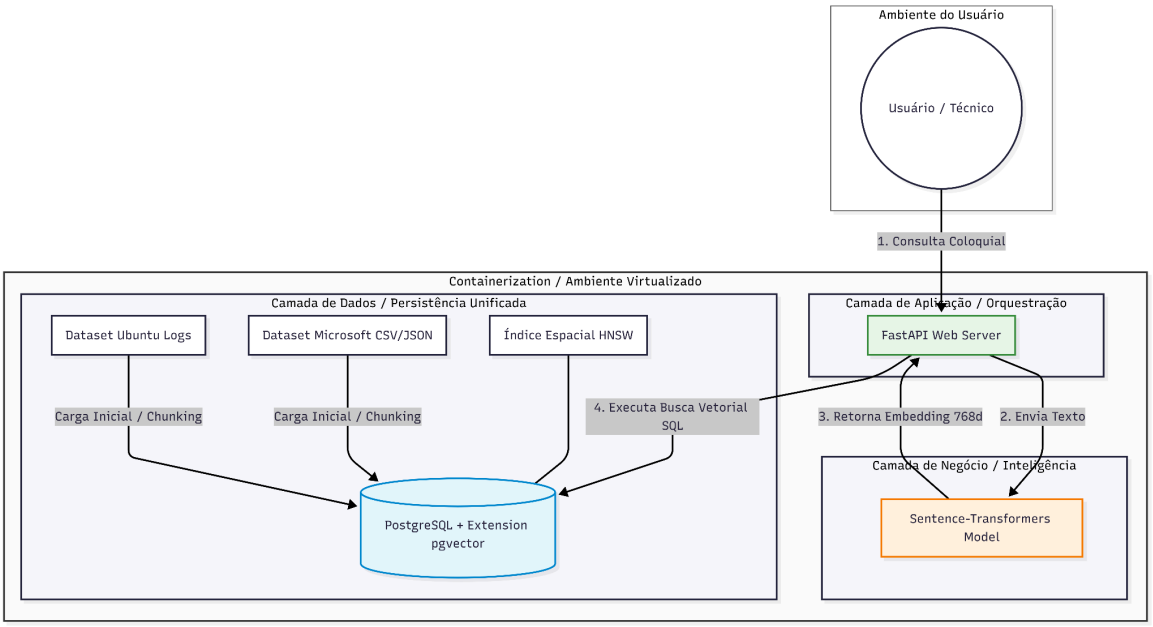


Figura 1 - Arquitetura de componentes e infraestrutura da solução
Fonte: Autoria própria (2026)

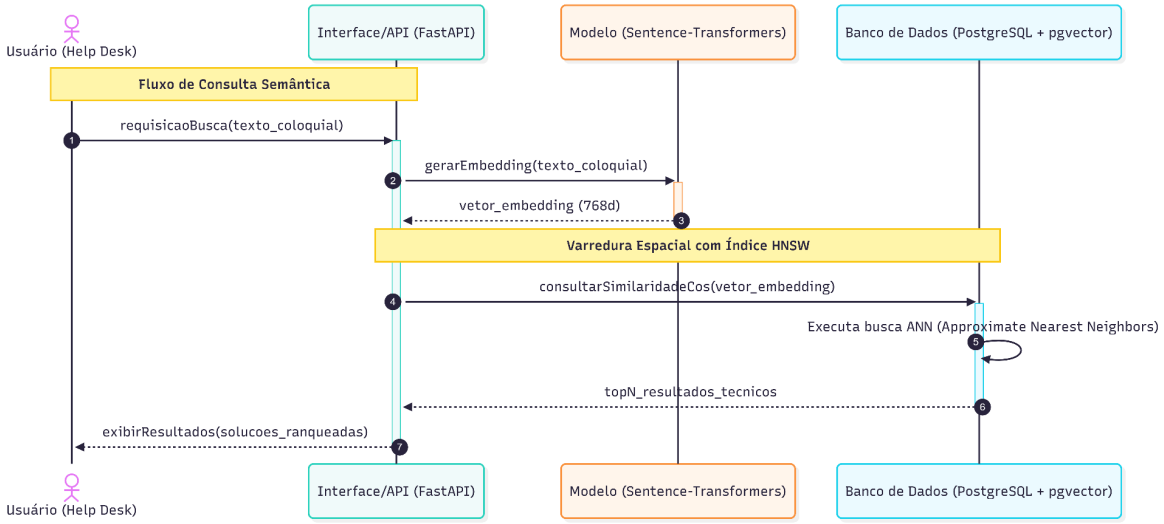


Figura 2 - Diagrama de fluxo de dados e orquestração da busca semântica
Fonte: Autoria própria (2026)

As funcionalidades detalhadas da ferramenta são apresentadas a seguir, categorizadas por seus respectivos módulos de atuação:

2.1.1 Módulo de Ingestão e Pré-processamento de Dados

- **Carga de Datasets Técnicos:** A ferramenta deve ser capaz de realizar a leitura e a extração de dados estruturados e semiestruturados (formatos JSON e CSV) provenientes de bases de conhecimento públicas e corporativas, especificamente os repositórios de suporte técnico da *Microsoft* e logs de diálogos do *Ubuntu*.
- **Sanitização e Fragmentação (*Chunking*):** O sistema deve aplicar técnicas de limpeza de texto (remoção de caracteres especiais e normalização) e segmentar os documentos longos em fragmentos menores (*chunks*). Esta segmentação é necessária para garantir que os embeddings gerados preservem a especificidade do contexto local de cada instrução técnica.

2.1.2 Módulo de Vetorização e Persistência Vetorial

- **Geração de Embeddings Textuais:** O sistema deve invocar modelos de linguagem pré-treinados da biblioteca *Sentence-Transformers* para mapear cada fragmento de texto sanitizado em um vetor denso de alta dimensionalidade.
- **Indexação Vetorial Otimizada:** A ferramenta deve interagir com o sistema de gerenciamento de banco de dados (SGBD) para indexar os vetores gerados utilizando estruturas de dados do tipo *Hierarchical Navigable Small World* (HNSW), otimizando o tempo de execução de buscas por vizinhos mais próximos (ANN - *Approximate Nearest Neighbors*).

2.1.3 Módulo de API e Orquestração de Consultas (Interface de Aplicação)

- **Interface de Busca Semântica:** A API (desenvolvida em *FastAPI*) deve expor um *endpoint* público para receber requisições de texto livre (consultas textuais dos usuários do *Help Desk*).
- **Mecanismo de Busca Lexical:** Como requisito de validação experimental, a ferramenta deve expor uma funcionalidade de busca tradicional baseada em correspondência exata de palavras-chave (utilizando recursos nativos de *Full-Text Search* do PostgreSQL), servindo como grupo de controle no experimento comparativo.
- **Cálculo de Similaridade e Ranqueamento:** O sistema deve converter a consulta em tempo real, computar a distância de cosseno no banco de dados e retornar um conjunto ranqueado contendo os *N* documentos com maior similaridade semântica.

Para sintetizar o escopo funcional e os limites da ferramenta propostos para o TCC 1, O Quadro 1 categoriza os requisitos essenciais do sistema.

Identificador	Nome da Funcionalidade	Descrição Sucinta	Prioridade
RF01	Ingestão de Datasets	Extração de dados estruturados de suporte da Microsoft e logs do Ubuntu em formatos CSV/JSON.	Obrigatório
RF02	Fragmentação de Texto	Segmentação de manuais técnicos extensos em chunks menores para preservação do contexto local.	Obrigatório
RF03	Geração de Embeddings	Invocação de modelos do Sentence-Transformers para conversão de texto em vetores densos (768d).	Obrigatório
RF04	Busca Vetorial (Semântica)	Execução de consultas por distância de cosseno utilizando índice HNSW mapeado no PostgreSQL.	Obrigatório
RF05	Busca Lexical (Tradicional)	Consulta por correspondência exata via Full-Text Search nativo do PostgreSQL (Grupo de Controle).	Obrigatório
RF06	Ranqueamento de Respostas	Ordenação e retorno do Top-N resultados mais similares à intenção da consulta do usuário.	Obrigatório

Quadro 1 - Matriz de requisitos funcionais do mecanismo de busca
Fonte: Autoria própria (2026)

2.2 Fundamentação Teórica em Recuperação da Informação e Modelagem Vetorial

A Recuperação da Informação (RI) é uma subárea da Ciência da Computação que estuda a localização de documentos de natureza não estruturada que satisfaçam a uma necessidade de informação declarada por um usuário. No contexto de engenharia de sistemas de suporte, os mecanismos de busca evoluíram de abordagens baseadas estritamente na sintaxe para modelos fundamentados na semântica textual.

2.2.1 Busca Lexical versus Busca Semântica

Os sistemas tradicionais de RI operam primordialmente por meio da busca lexical, utilizando algoritmos de correspondência exata de palavras-chave baseados no modelo vetorial booleano ou em esquemas de relevância estatística, como o TF-IDF e o BM25. Embora computacionalmente eficientes, essas abordagens sofrem com duas limitações linguísticas severas:

- **Sinonímia:** Ocorre quando o usuário expressa uma dúvida usando termos coloquiais ou sinônimos que não constam textualmente no manual técnico (ex: "computador travado" vs. "interrupção do sistema operacional").
- **Polissemia:** Ocorre quando uma mesma palavra assume significados distintos a depender do contexto em que está inserida.

Para mitigar a lacuna semântica, surge a busca baseada em representações densas (busca semântica). Em vez de rastrear a ocorrência exata de cadeias de caracteres, a abordagem semântica projeta o significado abstrato e o contexto das sentenças em um espaço geométrico contínuo.

2.2.2 Fundamentos de Embeddings e Modelos Sentence-Transformers

O processo de conversão de estruturas textuais discretas em representações numéricas contínuas é denominado geração de *embeddings*. Textos são mapeados como vetores densos em um espaço multidimensional de tamanho fixo D (onde $D = 768$ para a maioria das arquiteturas baseadas em BERT).

Diferente dos *word embeddings* tradicionais (como Word2Vec), que geram um vetor estático para cada palavra isolada, a arquitetura Sentence-Transformers utiliza redes neurais siamesas baseadas em *Transformers* (especificamente redes bi-encoder) para capturar o contexto de sentenças inteiras.

Matematicamente, dadas duas sentenças *A* e *B*, a rede processa cada uma individualmente, gerando os vetores *u* e *v.* A proximidade conceitual entre os textos é determinada pelo cálculo da Similaridade de Cosseno, definida pela equação (1):

$$\mathrm{Sim}(\vec{u},\vec{v}) = \frac{\vec{u}\cdot\vec{v}}{\|\vec{u}\|\|\vec{v}\|} = \frac{\sum_{i=1}^{D} u_i v_i}{\sqrt{\sum_{i=1}^{D} u_i^2}\sqrt{\sum_{i=1}^{D} v_i^2}}) \quad (1)$$

O resultado varia no intervalo [-1, 1], onde o valor 1 indica alinhamento semântico idêntico. Esse mecanismo matemático permite que o banco de dados recupere informações pela proximidade geométrica dos conceitos, superando a necessidade de correspondência de palavras-chave.

2.2.3 Formalização das Métricas de Avaliação

A validação de um sistema de RI exige a quantificação de sua capacidade de discriminar documentos relevantes de não relevantes. O embasamento teórico das métricas aplicadas neste experimento compreende:

- **Precisão (Precision):** Fração dos documentos recuperados pelo sistema que são verdadeiramente relevantes para a necessidade do usuário. É definida pela equação (2):

$$\text{Precisão} = \frac{|\{documentos\ relevantes\} \cap \{documentos\ recuperados\}|}{|\{documentos\ recuperados\}|} \quad (2)$$

- **Revocação (Recall):** Fração de todos os documentos relevantes presentes na base de dados que foram efetivamente capturados pelo sistema, expressa pela equação (3):

$$\text{Revocação} = \frac{|\{documentos\ relevantes\} \cap \{documentos\ recuperados\}|}{|\{documentos\ relevantes\ na\ base\}|} \quad (3)$$

- **Mean Reciprocal Rank (MRR):** Métrica focada na qualidade do ranqueamento posicional. Avalia a posição (posto ou *rank*) em que o primeiro documento considerado "padrão-ouro" (correto) foi retornado. Para um conjunto de Q consultas de teste, o MRR é calculado por:

$$\mathrm{MRR} = \frac{1}{|Q|}\sum_{i=1}^{Q}\frac{1}{\mathrm{posto}_i} \quad (4)$$

Onde $posto_i$ representa a posição ordinal da primeira resposta correta obtida para a consulta i.

2.3 Trabalhos Relacionados

A investigação de soluções correlatas na literatura é indispensável para identificar o estado da arte e demarcar o diferencial competitivo e científico da solução proposta. No domínio da Engenharia de Software voltada à Recuperação da Informação em bases de conhecimento, pesquisas recentes exploram a transição dos modelos lexicais para arquiteturas vetoriais.

Para contextualizar o posicionamento deste projeto, foram selecionados três trabalhos relevantes que abordam o uso de *embeddings*, busca semântica e a extensão *pgvector*:

1. **Batista, Costa e Jorge (2024) - Mecanismo de Busca Semântica Baseado em *Word Embeddings*:** Os autores implementaram um mecanismo de busca semântica para dados acadêmicos e do Currículo Lattes utilizando representações vetoriais para mitigar as falhas de correspondência exata de termos. Embora o sistema tenha otimizado a descoberta de competências, o escopo focou em um domínio acadêmico estruturado, deixando uma lacuna sobre a eficácia da abordagem quando aplicada a fluxos dinâmicos e conversacionais de suporte técnico corporativo (*Help Desk*).
2. **Silva *et al.* (2024) - *A Comparative Study of BERT Models for Semantic Retrieval*:** Esta pesquisa avaliou o desempenho de modelos baseados em BERT para a tarefa de similaridade textual e recuperação de precedentes jurídicos. O estudo contribui ao detalhar estratégias avançadas de divisão de documentos extensos em fragmentos menores (*chunking*), porém limitou-se ao domínio do direito e adotou o ecossistema *Elasticsearch* para a persistência vetorial, deixando em aberto a validação de uma arquitetura relacional clássica e unificada.
3. **Lauton (2025) - *DiveScan*:** um módulo para Recuperação de dados por Similaridade em Postgres: A pesquisadora desenvolveu e avaliou um módulo de extensão integrado nativamente ao ambiente do PostgreSQL para acelerar a busca espacial por vizinhos mais próximos. O trabalho focou estritamente em aspectos de baixo nível e na eficiência de indexação estrutural (como o uso de índices HNSW), carecendo de uma validação voltada à qualidade e relevância das respostas textuais sob a ótica do usuário final por meio de métricas consolidadas como o MRR.

O Quadro 2 sintetiza a comparação entre os trabalhos correlatos e a abordagem projetada para este TCC 1, evidenciando as lacunas existentes na literatura prática.

Critério de Comparação	Batista, Costa e Jorge (2024)	Silva et al. (2024)	Lauton (2025)	Proposta Deste Projeto
Modelo de Embeddings	Vetores de Palavras (*Word Embeddings*)	Locais Especializados (BERT)	Não aplicável (Foco em Banco)	Local (*Sentence-Transformers*)
Persistência Vetorial	Indexação em Base de Dados	Arquivo Dedicado (*Elasticsearch*)	Relacional Estendido (Postgres)	Relacional unificado (pgvector)
Foco da Investigação	Contexto de dados acadêmicos	Estratégia de *chunking* de texto	Algoritmos e Índices HNSW	Arquitetura de dados e Relevância
Métricas de Validação	Relevância e eficácia de busca	Métricas de recuperação de textos	Eficiência de busca espacial	Quantitativo (*Precision*, *Recall*, MRR)

Quadro 2 - Matriz comparativa de trabalhos relacionados
Fonte: Autoria própria (2026)

O diferencial desta proposta frente à literatura revisada consolida-se na unificação de três pilares: o uso de uma infraestrutura relacional de baixo custo e alta consistência (*pgvector*), a independência tecnológica por meio de modelos *open-source* locais (*Sentence-Transformers*) e a imposição de um desenho experimental de alto rigor matemático (*Precision, Recall e MRR*) voltado especificamente a simular o comportamento de usuários leigos em ambientes de *Help Desk*.

2.4 Persistência de Dados

A arquitetura de persistência de dados desempenha um papel fundamental na sustentação de sistemas de recuperação de informação, sendo responsável por garantir a integridade, o desempenho e a escalabilidade no armazenamento de dados estruturados e vetoriais (NAVATHE; ELMASRI, 2018). Para o mecanismo de busca semântica proposto, a camada de persistência foi projetada para unificar o armazenamento de metadados relacionais

e vetores densos de alta dimensionalidade em uma única infraestrutura de banco de dados, utilizando o SGBD *PostgreSQL* acoplado à extensão *pgvector*.

Diferente das abordagens que fragmentam a arquitetura utilizando bancos de dados vetoriais dedicados (como *Pinecone* ou *Milvus*), a escolha por uma solução relacional estendida justifica-se pela necessidade de manter a integridade referencial forte e simplificar o pipeline de dados. O modelo de dados especificado para o ecossistema de *Help Desk* é composto por estruturas que mapeiam os documentos originais, seus respectivos fragmentos (*chunks*) de texto e os vetores calculados pelo modelo de linguagem.

A seguir, apresenta-se a modelagem das estruturas de dados projetadas para o sistema por meio do esquema das tabelas.

Tabela	**Atributo**	**Tipo de Dados**	**Restrições / Descrição**
documentos	id	SERIAL	PRIMARY KEY - Identificador do manual ou FAQ original.
	titulo	VARCHAR(255)	NOT NULL - Título do documento de suporte técnico.
	fonte	VARCHAR(50)	Nome do dataset de origem (*Microsoft* ou *Ubuntu*).
documento_chunks	id	SERIAL	PRIMARY KEY - Identificador do fragmento.
	documento_id	INTEGER	FOREIGN KEY - Relacionamento com a tabela documentos.

	conteudo	TEXT	NOT NULL - Texto limpo do fragmento de manual técnico.
	embedding	VECTOR(768)	Armazenamento do vetor denso gerado pelo modelo (768 dimensões).

Quadro 3 - Dicionário de dados das tabelas de persistência vetorial
Fonte: Autoria própria (2026)

Para viabilizar a implementação do modelo projetado e garantir o rigor científico exigido na manipulação de estruturas vetoriais, a especificação técnica do banco de dados estabelece três etapas de comandos DDL (*Data Definition Language*) e DML (*Data Manipulation Language*) fundamentais:

1. Habilitação do Módulo Vetorial: A extensão deve ser inicializada no servidor de banco de dados para injetar o tipo nativo VECTOR e os operadores geométricos espaciais:

```
CREATE EXTENSION IF NOT EXISTS pgvector;
```

2. Criação do Índice de Busca Otimizada (HNSW): Para acelerar a recuperação de informações sem realizar uma varredura linear (*sequential scan*) em toda a base de dados, especifica-se a criação de um índice do tipo *Hierarchical Navigable Small World* (HNSW). O índice é configurado utilizando a métrica de distância de cosseno (vector_cosine_ops), que avalia a similaridade angular entre os vetores independentemente da magnitude do texto:

```
CREATE INDEX idx_chunks_embedding_hnsw
ON documento_chunks
USING hnsw (embedding vector_cosine_ops);
```

3. Orquestração da Consulta Semântica via SQL: A recuperação dos *N* documentos mais relevantes baseados na intenção de busca do usuário é modelada através de uma consulta que computa a distância de cosseno por meio do operador nativo <=>. A consulta calcula a proximidade geométrica entre o vetor da pergunta enviada pela API e os embeddings armazenados, ordenando os resultados de forma ascendente (onde a menor distância representa a maior similaridade):

```
SELECT documento_id, conteudo, (embedding <=> :vetor_consulta) AS
distancia
```

```
FROM documento_chunks
ORDER BY embedding <=> :vetor_consulta
LIMIT :top_n;
```

Essa modelagem de persistência unificada assegura que as consultas textuais executadas pela camada de aplicação usufruam de alta performance e baixo tempo de resposta, viabilizando a validação futura do experimento frente às métricas de desempenho e relevância exigidas.

2.5 Tecnologias Utilizadas

A seleção do ecossistema tecnológico para este projeto baseou-se em critérios de robustez, interoperabilidade e adequação aos requisitos de Engenharia de Dados e Processamento de Linguagem Natural (PLN). Como o objetivo central reside na modelagem de uma arquitetura unificada para recuperação de informações , priorizou-se o uso de ferramentas consagradas e de código aberto. As principais tecnologias especificadas e suas respectivas justificativas de projeto são descritas a seguir:

- **Linguagem Python:** Escolhida por ser a linguagem padrão da indústria para o desenvolvimento de soluções de Inteligência Artificial e ciência de dados. Sua vasta disponibilidade de bibliotecas ecossistêmicas viabiliza a manipulação eficiente de matrizes numéricas e strings.
- **FastAPI:** *Framework* web assíncrono de alto desempenho utilizado para a especificação das interfaces de programação de aplicações (APIs). A escolha justifica-se pela sua baixa latência no processamento de requisições, documentação automática nativa via *Swagger* e validação estrita de tipos de dados por meio da biblioteca *Pydantic*, garantindo a resiliência no recebimento de consultas em tempo real.
- **Sentence-Transformers:** Biblioteca de aprendizado de máquina baseada na arquitetura *Transformers* utilizada para computar representações vetoriais densas (*embeddings*) a partir de fragmentos textuais técnicos. Justifica-se pelo uso de modelos pré-treinados otimizados para tarefas de busca semântica, mapeando contextos complexos em vetores de 768 dimensões com alta fidelidade conceitual.
- **PostgreSQL e pgvector:** O sistema de gerenciamento de banco de dados relacional foi adotado devido à sua confiabilidade histórica em ambientes produtivos. O uso da extensão *pgvector* é a peça central da arquitetura de persistência, permitindo operar dados vetoriais de forma nativa por meio de comandos SQL clássicos , eliminando a necessidade de gerenciar infraestruturas externas e apartadas para os índices espaciais.

2.6 Desenho Experimental e Protocolo de Avaliação

Para validar a eficácia do mecanismo de busca semântica proposto em relação à abordagem lexical tradicional, o experimento adotará um modelo de avaliação baseado em conjuntos de testes controlados. Conforme apontado por Pressman e Maxim (2021), a validação de uma ferramenta técnica deve ser sustentada por métricas quantitativas mensuráveis. No domínio da Recuperação da Informação (RI), o rigor científico do experimento será garantido pela aplicação de três métricas clássicas sobre os resultados retornados pelas consultas: **Precisão@5**, **Revocação@5** e a **Média do Posto Recíproco (MRR)**, cujas formulações matemáticas foram detalhadas na Seção 2.2.

O objetivo deste arranjo experimental é mensurar, de forma estatística e reprodutível, o ganho de relevância e exatidão proporcionado pela modelagem vetorial em cenários reais de atendimento técnico.

2.6.1 Definição da Amostra e Conjunto de Dados

O experimento utilizará uma abordagem de teste controlada dividida em dois cenários distintos extraídos dos datasets descritos no escopo do projeto:

- **Cenário A (Suporte Microsoft):** Composto por uma amostragem aleatória de 100 documentos de FAQs corporativas. Sobre essa base, serão geradas 30 consultas de teste simulando usuários reais.
- **Cenário B (Logs Ubuntu):** Composto por uma amostragem de 100 diálogos técnicos de suporte a sistemas operacionais, sobre a qual serão aplicadas outras 30 consultas de teste.

No total, o protocolo de avaliação será submetido a um conjunto de 60 consultas analíticas, garantindo relevância estatística para o cálculo das médias das métricas estabelecidas.

2.6.2 Estabelecimento do Padrão-Ouro (*Ground Truth*)

Para calcular com precisão as métricas de *Precision* e *Recall*, é necessário determinar previamente quais documentos são verdadeiramente relevantes para cada uma das 60 consultas de teste. Este mapeamento será consolidado por meio de uma abordagem de análise de especialistas.

Cada consulta terá seus documentos correspondentes anotados em uma matriz de gabarito (padrão-ouro). Um documento será classificado como "Relevante" (valor binário 1) se contiver explicitamente a solução para o problema descrito na consulta coloquial, e "Não Relevante" (valor binário 0) caso contrário.

2.6.3 Pipeline de Execução dos Testes

O fluxo de execução do experimento ocorrerá de forma automatizada por meio de scripts de teste em *Python*, seguindo o protocolo linear descrito a seguir:

1. **Disparo das Consultas:** O script enviará sequencialmente as consultas de teste para dois ambientes isolados: o motor de busca lexical nativo do PostgreSQL e o endpoint da API de busca semântica (*FastAPI* + *pgvector* com índice HNSW).
2. **Coleta de Rankings:** Para cada consulta, ambos os sistemas retornarão um ranking contendo o *Top-5* ($N = 5$) documentos mais similares recuperados.
3. **Cálculo Automatizado:** O script confrontará os IDs dos documentos retornados em cada posição do ranking contra a matriz de padrão-ouro. A partir desse cruzamento, serão computados os valores exatos de Precisão@5, Revocação@5 e o posto recíproco para o cálculo do MRR.

Os dados coletados serão tabulados e submetidos a testes de hipóteses estatísticas no ciclo do TCC 2, permitindo avaliar se a diferença de desempenho entre as soluções possui significância e justificativa técnica sob a ótica da Engenharia de Software.

2.7 Etapas e Cronograma de Desenvolvimento

O planejamento temporal e a distribuição de atividades são fundamentais para assegurar a viabilidade de execução do projeto dentro dos prazos acadêmicos estipulados para o ciclo do TCC 2. Conforme preconizado pelas metodologias ágeis de desenvolvimento de software (PRESSMAN; MAXIM, 2021), as atividades foram estruturadas de forma incremental, garantindo que o acoplamento entre os módulos projetados ocorra de maneira controlada.

O desenvolvimento da ferramenta proposto foi dividido em cinco macroetapas sequenciais, detalhadas a seguir:

1. **Configuração e Infraestrutura:** Criação do ambiente virtualizado utilizando *Docker* para isolar a instância do SGBD *PostgreSQL* e configuração das dependências iniciais da API em *FastAPI*.
2. **Pipeline de Ingestão e Vetorização:** Implementação dos scripts em *Python* para leitura dos conjuntos de dados extraídos do suporte da *Microsoft* e do *Ubuntu*, seguida do processo de fragmentação (*chunking*) e integração com a biblioteca *Sentence-Transformers* para carga inicial do banco vetorial.
3. **Mecanismos de Busca e Otimização:** Configuração dos índices *HNSW* no banco de dados com a extensão *pgvector* e codificação das rotas da API para orquestrar as consultas em tempo real (tanto para a abordagem semântica quanto para a lexical).

4. **Execução do Experimento e Coleta de Métricas:** Submissão das baterias de testes automatizados conforme o protocolo definido no desenho experimental, mensurando e registrando os índices de **Precisão@5, Revocação@5 e MRR** obtidos por cada modelo de busca.
5. **Análise de Resultados e Redação Final:** Consolidação estatística dos dados coletados, comparação analítica estruturada entre as abordagens e escrita da monografia final para a defesa do TCC 2.

O Quadro 4 apresenta a distribuição cronológica dessas atividades estimada para o próximo período letivo.

Macroetapa / Atividade	**Julho**	**Agosto**	**Setembro**	**Outubro**	**Novembro**
Etapa 1: Configuração do Ambiente e Docker	**X**				
Etapa 2: Ingestão, *Chunking* e Carga de Datasets	**X**	**X**			
Etapa 3: Criação de Índices HNSW e Endpoints FastAPI		**X**	**X**		
Etapa 4: Testes e Coleta de Métricas (Precision/Recall/MRR)			**X**	**X**	
Etapa 5: Análise Estatística e Redação do Relatório Final				**X**	**X**

Quadro 4 - Cronograma de atividades projetado para o TCC 2
Fonte: Autoria própria (2026)

3 CONSIDERAÇÕES FINAIS

A elaboração desta proposta de trabalho de conclusão de curso permitiu estruturar e especificar, sob o rigor da Engenharia de Software, um mecanismo de busca semântica unificado voltado a otimizar o atendimento em bases de conhecimento de *Help Desk*. Diante das limitações identificadas nos sistemas tradicionais baseados em busca lexical, o design arquitetural proposto demonstrou-se viável ao integrar o modelo de linguagem *Sentence-Transformers* à robustez relacional do *PostgreSQL* com a extensão *pgvector*.

Todas as metas estipuladas para esta etapa inicial foram plenamente alcançadas. Realizou-se a especificação detalhada dos requisitos funcionais do sistema, a modelagem conceitual do dicionário de dados da persistência vetorial e o desenho de diagramas complementares que evidenciam as perspectivas estática e dinâmica da solução tecnológica. Além disso, a fundamentação teórica construída e o mapeamento da literatura correlata permitiram delimitar com clareza o diferencial científico e a relevância prática deste projeto no domínio da recuperação da informação.

Por tratar-se de uma proposta de pesquisa, o escopo deste relatório limitou-se ao planejamento e design da solução. Como desdobramentos futuros e passos imediatos estabelecidos no cronograma para o próximo período letivo (TCC 2), serão executadas as fases de implementação física da infraestrutura em *Docker*, o desenvolvimento das rotas assíncronas da API em *FastAPI* e a automação da pipeline de ingestão dos dados brutos.

Por fim, o núcleo prático do próximo ciclo se concentra na condução do desenho experimental especificado, submetendo o protótipo funcional a baterias de testes controlados. A coleta automatizada e a análise estatística das métricas de Precisão@5, Revocação@5 e *Mean Reciprocal Rank* (MRR) serão fundamentais para validar a eficácia da inteligência algorítmica proposta, consolidando a entrega de um produto de software robusto, escalável e de alto valor agregado para o gerenciamento de serviços de TI.

REFERÊNCIAS

BATISTA, J. V. C. R.; COSTA, G. M.; JORGE, E. M. F. Mecanismo de Busca Semântica Baseado em Word Embeddings em Dados do Currículo Lattes, Programas de Pós-Graduação e Grupos de Pesquisa. In: ERBASE, 24., 2024, Bahia. **Anais** [...]. Porto Alegre: SBC, 2024. p. 109-118. Disponível em: <https://sol.sbc.org.br/index.php/erbase/article/view/33882>. Acesso em: 26 maio 2026.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2018.

LAUTON, A. J. C. DiveScan: um módulo para Recuperação de dados por Similaridade com Diversidade em Postgres. In: SBBD, 40., 2025, Salvador. **Anais** [...]. Porto Alegre: SBC, 2025. p. 1-13. Disponível em: <https://sol.sbc.org.br/index.php/sbbd/article/view/37297>. Acesso em: 01 jun. 2026.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software: uma abordagem profissional**. 9. ed. Porto Alegre: AMGH, 2021.

SILVA, C. *et al*. A Comparative Study of BERT Models for Semantic Retrieval of Brazilian Legal Precedents. In: KDMILE, 12., 2024, Curitiba. **Anais** [...]. Porto Alegre: SBC, 2024. p. 65-72. Disponível em: <https://sol.sbc.org.br/index.php/kdmile/article/view/37210>. Acesso em: 01 jun. 2026.